

Diagrama De Flujo DelCodigo technical_V.E.S.T.A

Mapa completo del flujo entre UI web, servidor local, firmware ESP32-S3, firmware ESP32-CAM, serial, WebSocket, BLE, video y perfil de calibracion.

app.js

server.js

login.js

ESP32-S3

ESP32-CAM

WebSocket / Serial / BLE

Proyecto: C:\VSC\technical_V.E.S.T.A

Generado: 28/05/2026

Formato: A4 horizontal, diagramas Graphviz renderizados localmente.

Contenido

1. 1. Arquitectura General Del Sistema
2. 2. Login, Sesion Y Entrada A La Consola
3. 3. Inicializacion De La Consola Web
4. 4. Navegacion Y Paneles De La Consola
5. 5. Conexion, Descubrimiento Y Telemetria
6. 6. Servidor Local: Rutas API Y Archivos
7. 7. Carga De Firmware Con arduino-cli
8. 8. Puente Serial Y Eventos SSE
9. 9. ESP32-S3: Setup Y Loop Principal
10. 10. ESP32-S3: Sensores, Servos Y Seguridad
11. 11. ESP32-S3: Comandos JSON Y BLE
12. 12. ESP32-CAM: Setup, Video, Audio Y Puente Al S3
13. 13. Flujo De Video En Banco: USB MJPEG Y Fallback HTTP
14. 14. Perfil De Calibracion Y Persistencia
15. 15. Readiness, Pruebas Y Seguridad Operativa
16. 16. Mapa De Archivos Y Responsabilidad
17. 17. Entradas Y Salidas Principales
18. 18. Comandos JSON Relevantes

Lectura rapida

Cada pagina tiene un objetivo y un grafo con nodos principales, decisiones y rutas de datos.

Colores

Azul: UI. Naranja: herramientas/servidor. Verde: hardware/estado OK. Rojo: seguridad o fallas.

Mapa De Archivos Y Responsabilidad

Elemento	Rol dentro del flujo	Archivo principal
Login	Validacion local, bloqueo de intentos y sesion	index.html, login.js, login.css, credentials.js
Consola tecnica	UI de armado, diagnostico, firmware, servos, sensores, pruebas y entrega	technical_V.E.S.T.A..html, app.js, styles.css
Servidor local	API HTTP, carga de firmware, serial bridge, proxy CAM y archivos estaticos	server.js
Firmware S3	Control en tiempo real de sensores, servos, BLE, WebSocket y seguridad	esp32_s3_controller.ino, esp32_s3_config.h
Firmware CAM	Video MJPEG, USB MJPEG, audio I2S, estado y puente hacia S3	esp32_cam_assistant.ino, esp32_cam_config.h
Documentacion	Uso, firmware, protocolos y contexto tecnico	README.md, FIRMWARE.md

Entradas Y Salidas Principales

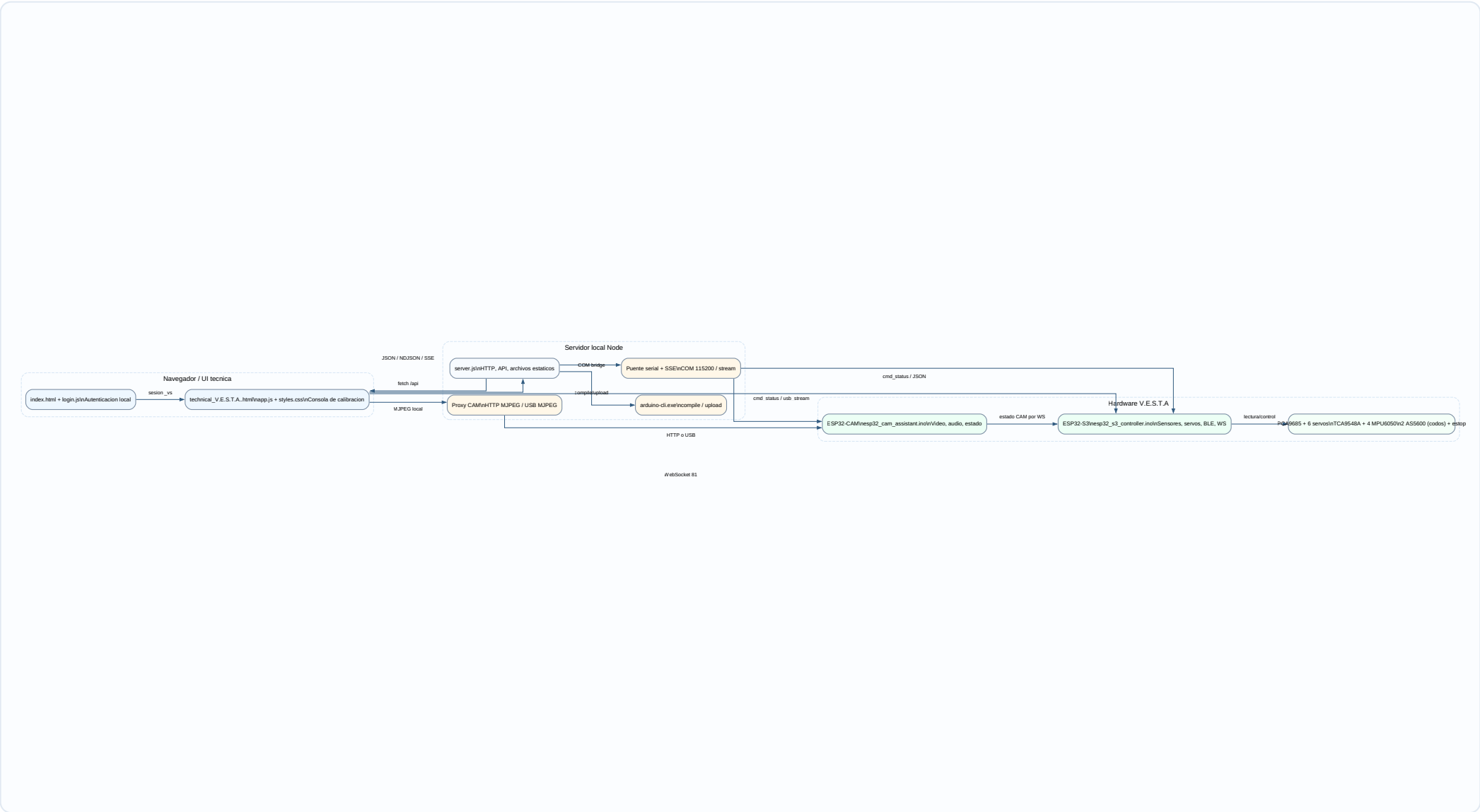
Elemento	Rol dentro del flujo	Archivo principal
Usuario tecnico	Cambia paneles, calibra, prueba, sube firmware y envia perfil	app.js
Navegador	WebSocket, fetch JSON, NDJSON, SSE, localStorage y canvas	app.js
Servidor Node	Responde API, ejecuta arduino-cli y abre puentes seriales	server.js
ESP32-S3	Recibe cmd_*, publica telemetria, mueve servos y guarda preferencias	esp32_s3_controller.ino
ESP32-CAM	Entrega video/audio/estado por HTTP, WS o serial USB	esp32_cam_assistant.ino
Hardware fisico	MPU6050 (hombros), AS5600 (codos), PCA9685, servos, boton de emergencia y red WiFi/BLE	*.h, *.ino

Comandos JSON Relevantes

Elemento	Rol dentro del flujo	Archivo principal
cmd_status	Solicita estado completo, IP, URLs, modo y telemetria	app.js -> esp32_s3_controller.ino
cmd_mode	Cambia manual/assisted/automatic si no hay emergencia	app.js -> processCmd()
cmd_angle	Define targetDeg[id] para control manual o prueba	sendAngle() -> processCmd()
cmd_assist	Ajusta asistencia 0..1 y persiste preferencia	sendMapProfile() -> savePrefs()
cmd_calibration_profile	Entrega perfil completo de servos, IMU, AS5600 y tuning	buildProfile() -> applyCalibrationProfile()
cmd_stop / cmd_reset / cmd_home	Seguridad, reset seguro y retorno a home	sendCommand() -> setEmergency()/writeServo()

1. Arquitectura General Del Sistema

Todos los componentes del sistema y como se conectan: la consola web del tecnico (navegador), el servidor Node.js corriendo en la PC del tecnico, el ESP32-S3 que maneja sensores/servos/BLE, y la ESP32-CAM que provee video. Las flechas muestran que protocolo usa cada conexion (WebSocket, fetch /api, MJPEG, serial COM).



2. Login, Sesión Y Entrada A La Consola

El tecnico abre index.html. login.js carga las credenciales almacenadas, hashea la contraseña con SHA-256 (Web Crypto) y la compara. Tras 5 intentos fallidos se activa un bloqueo de 15 minutos. Si la contraseña es correcta se genera un token de sesión en sessionStorage y se redirige a la consola principal.



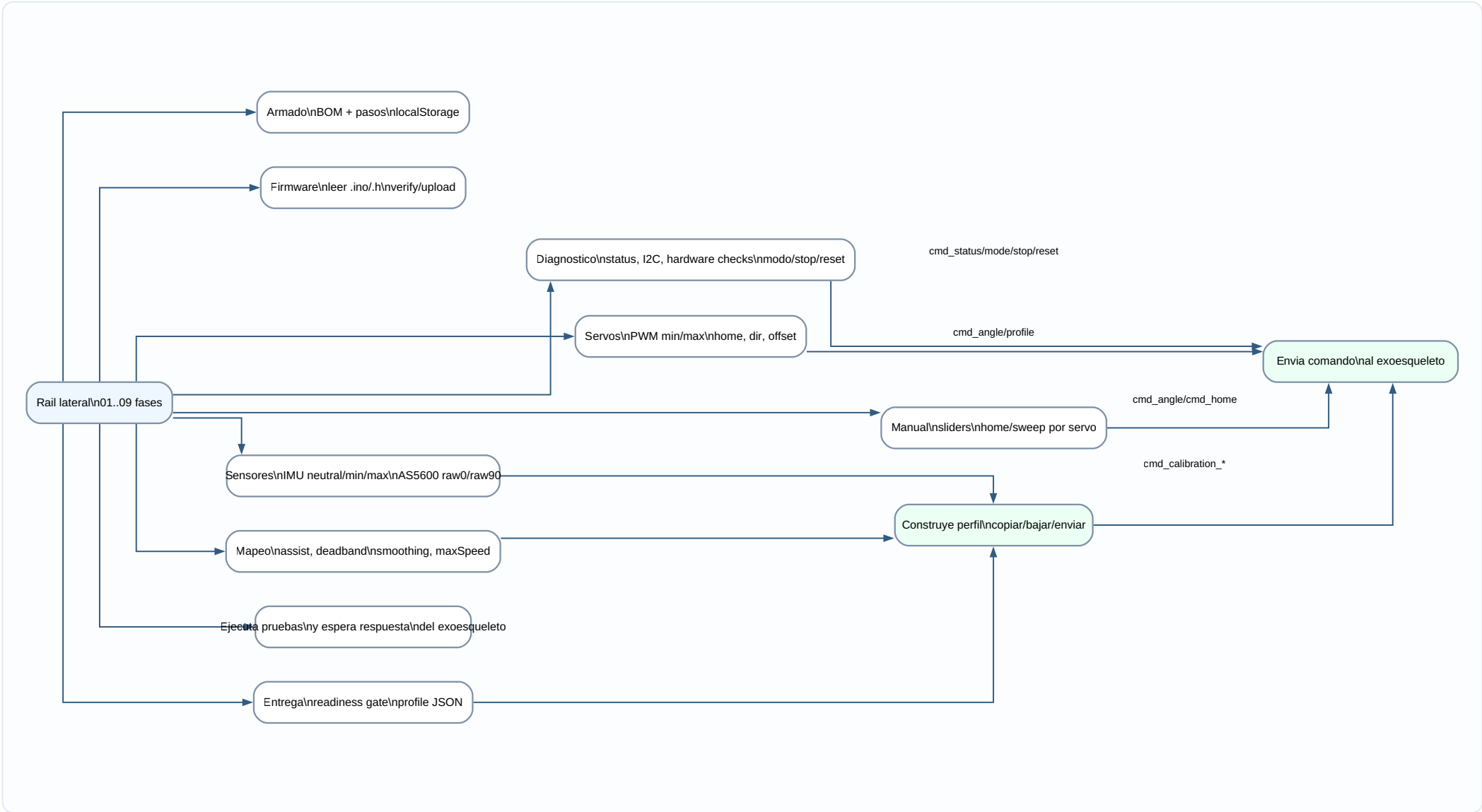
3. Inicializacion De La Consola Web

app.js arranca en un IIFE (modulo inmediato): carga constantes de hardware (SERVO_DEFAULTS, IMU_DEFAULTS), restaura el estado guardado en localStorage (defaultState → loadState → hydrateState), construye todos los paneles de la UI, enlaza botones/formularios, descarga el firmware bundled y queda listo para recibir telemetria del ESP32-S3.



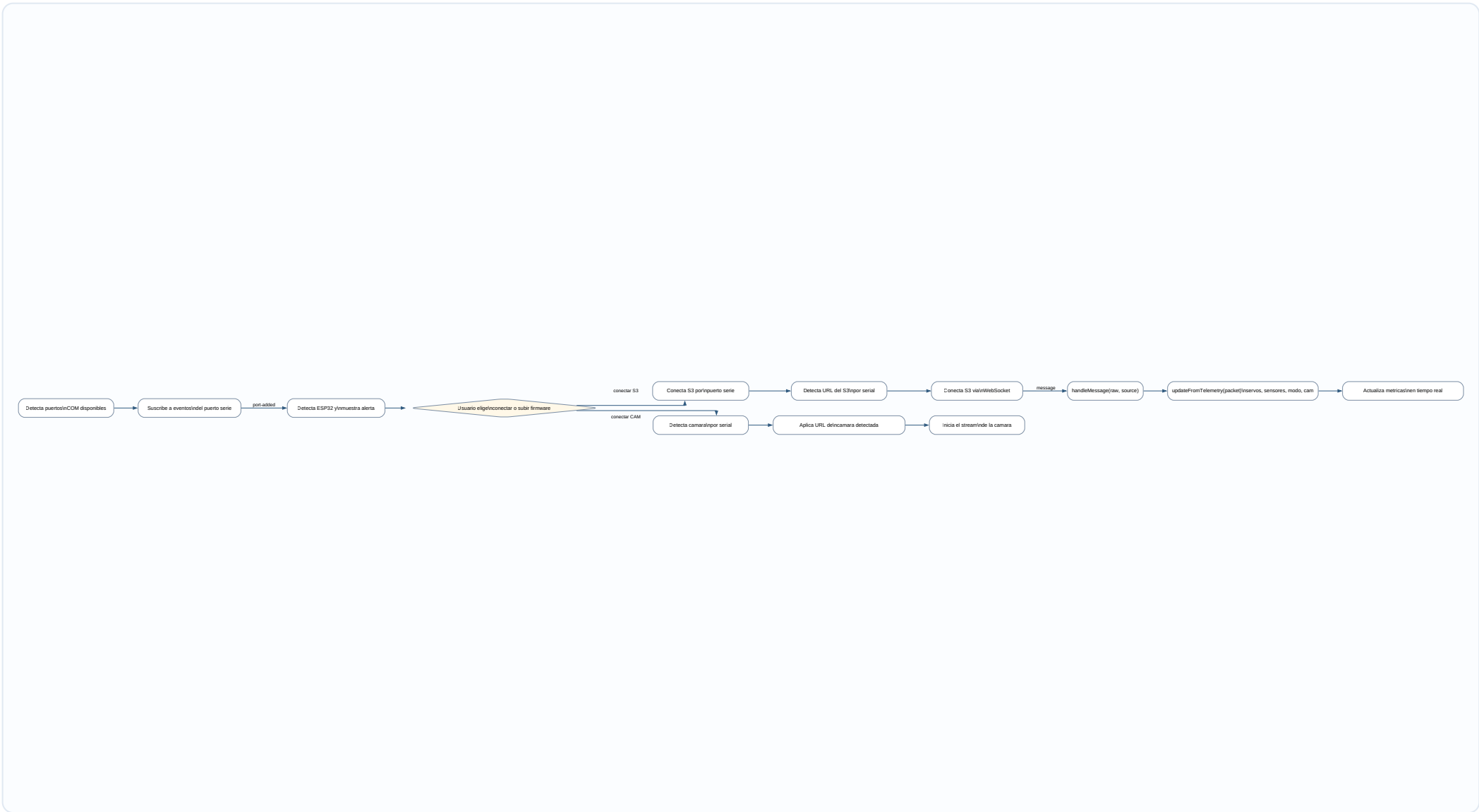
4. Navegacion Y Paneles De La Consola

La barra lateral (Rail) da acceso a 9 paneles. Armado y Firmware son de preparacion. Diagnostico, Servos, Manual, Sensores y Mapeo son de calibracion: cada uno envia comandos JSON al ESP32-S3 o acumula datos para el perfil. Pruebas verifica el hardware en vivo. Entrega construye y envia el perfil completo al exoesqueleto.



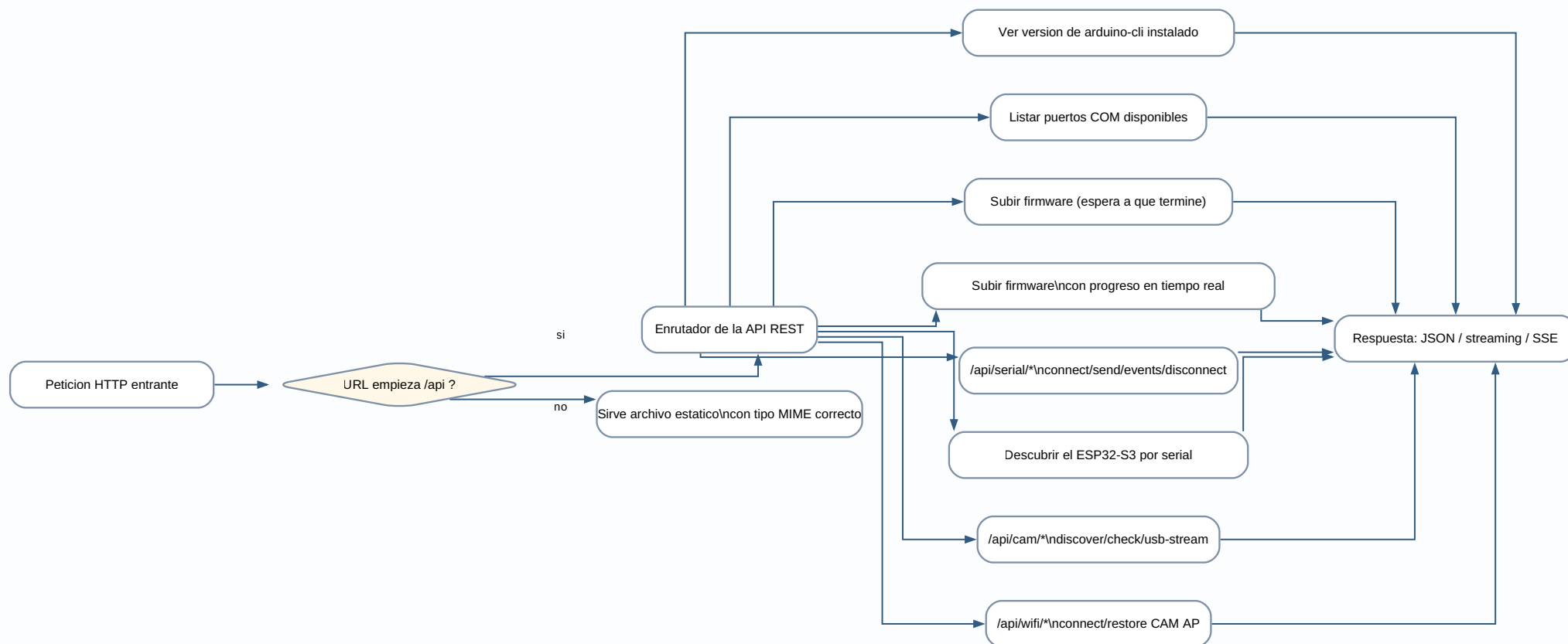
5. Conexion, Descubrimiento Y Telemetria

Al detectar un nuevo puerto COM o al pulsar Conectar, la app busca automaticamente el ESP32-S3 y la ESP32-CAM. Para el S3: abre el COM, envia cmd_status y lee la IP. Para la CAM: escanea puertos o detecta la URL de red. Una vez conectado, el WebSocket recibe telemetria en tiempo real y actualiza la UI (angulos, sensores, modo, video).



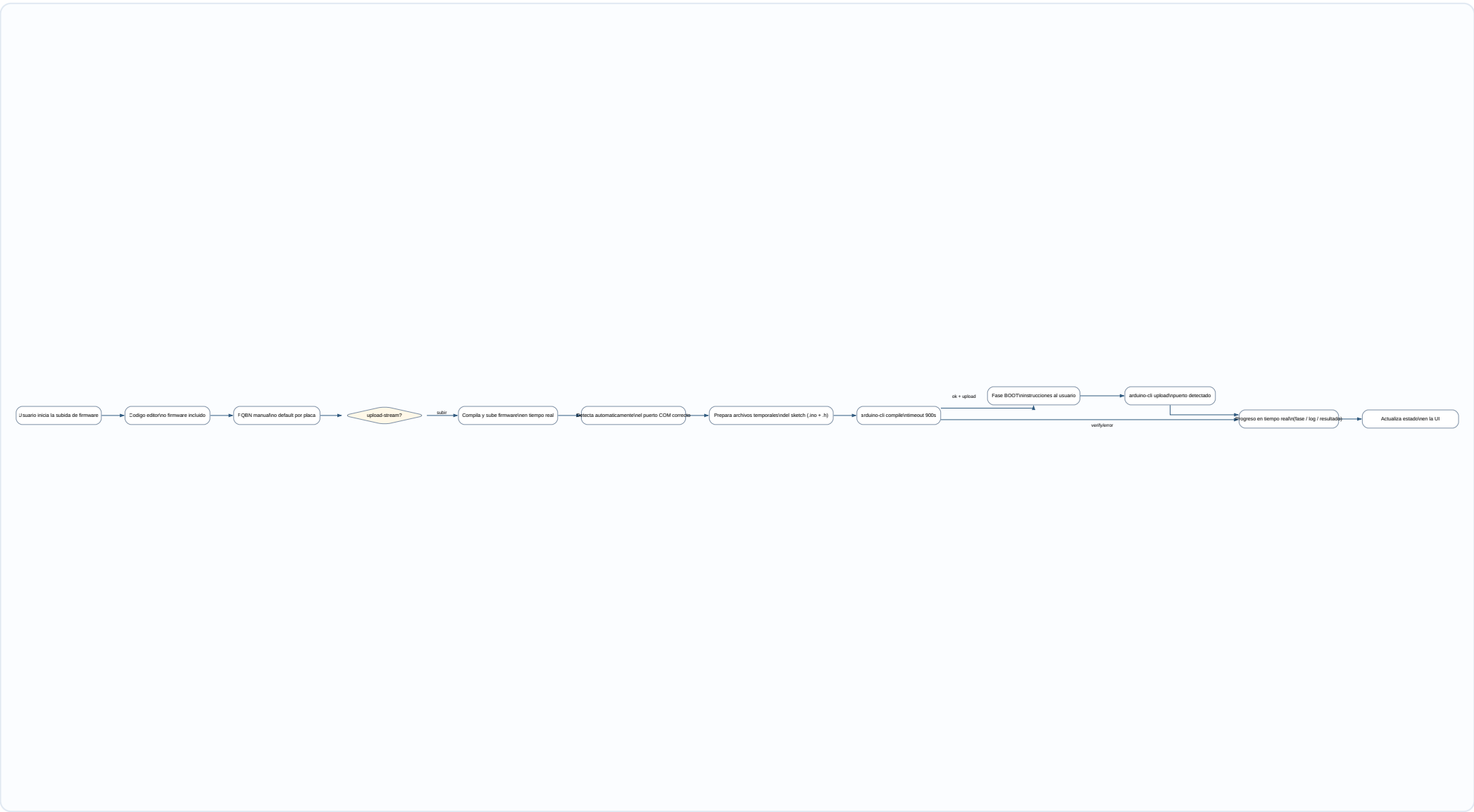
6. Servidor Local: Rutas API Y Archivos

server.js actua como gateway HTTP unico (puerto 5177+). Su funcion handler() enruta cada peticion: las rutas /api/* van al apiHandler() que despacha segun el pathname; los archivos estaticos se sirven desde disco con proteccion anti path-traversal (safePath()). Rutas clave: /api/arduino/ports y /api/arduino/upload-stream para subir firmware; /api/serial/* para el puente COM (conectar, enviar, escuchar eventos, desconectar); /api/cam/* para descubrir y transmitir la ESP32-CAM; /api/s3/discover para localizar el ESP32-S3 por serial; /api/wifi/* para gestionar el AP de emergencia. Las respuestas usan sendJson() (JSON unico), sendNdjson() (lineas JSON en tiempo real durante la compilacion) o sendSse() (Server-Sent Events para el monitor serial).



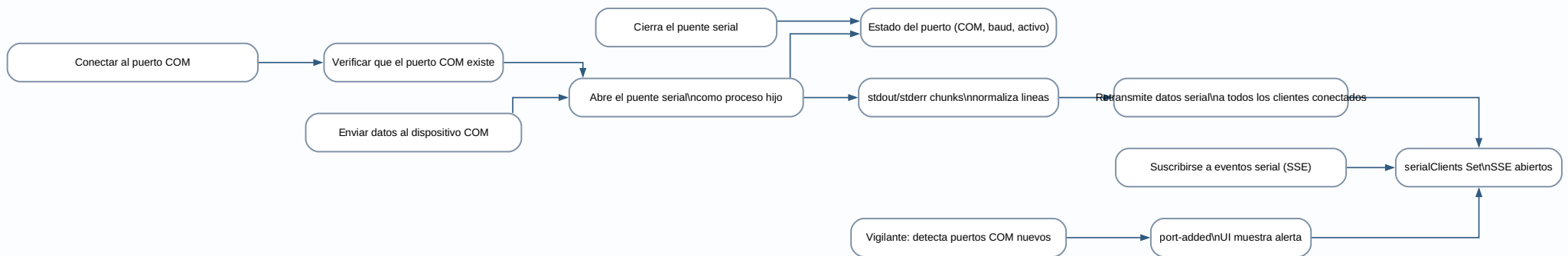
7. Carga De Firmware Con arduino-cli

Al pulsar Subir Firmware, runFirmwareAction() recopila el código del editor o usa el firmware incluido y determina el FQBN (fully qualified board name) de la placa. handleUploadStream() recibe el POST, detecta automáticamente el puerto COM correcto puntuando los puertos disponibles con detectUploadTarget(), copia el sketch y archivos .h a un directorio temporal con writeSketchTemp(), y lanza arduino-cli compile (timeout 900 s). Si compila OK, entra la fase BOOT: la UI indica al usuario mantener BOOT presionado mientras arduino-cli upload escribe el .bin al chip. El progreso se transmite en tiempo real al navegador como eventos NDJSON via fetchNdjson(): cada línea JSON lleva fase (compiling/uploading/done) y el log de salida. Si hay error, el evento verify/error detiene el flujo y muestra el mensaje en la consola de firmware.



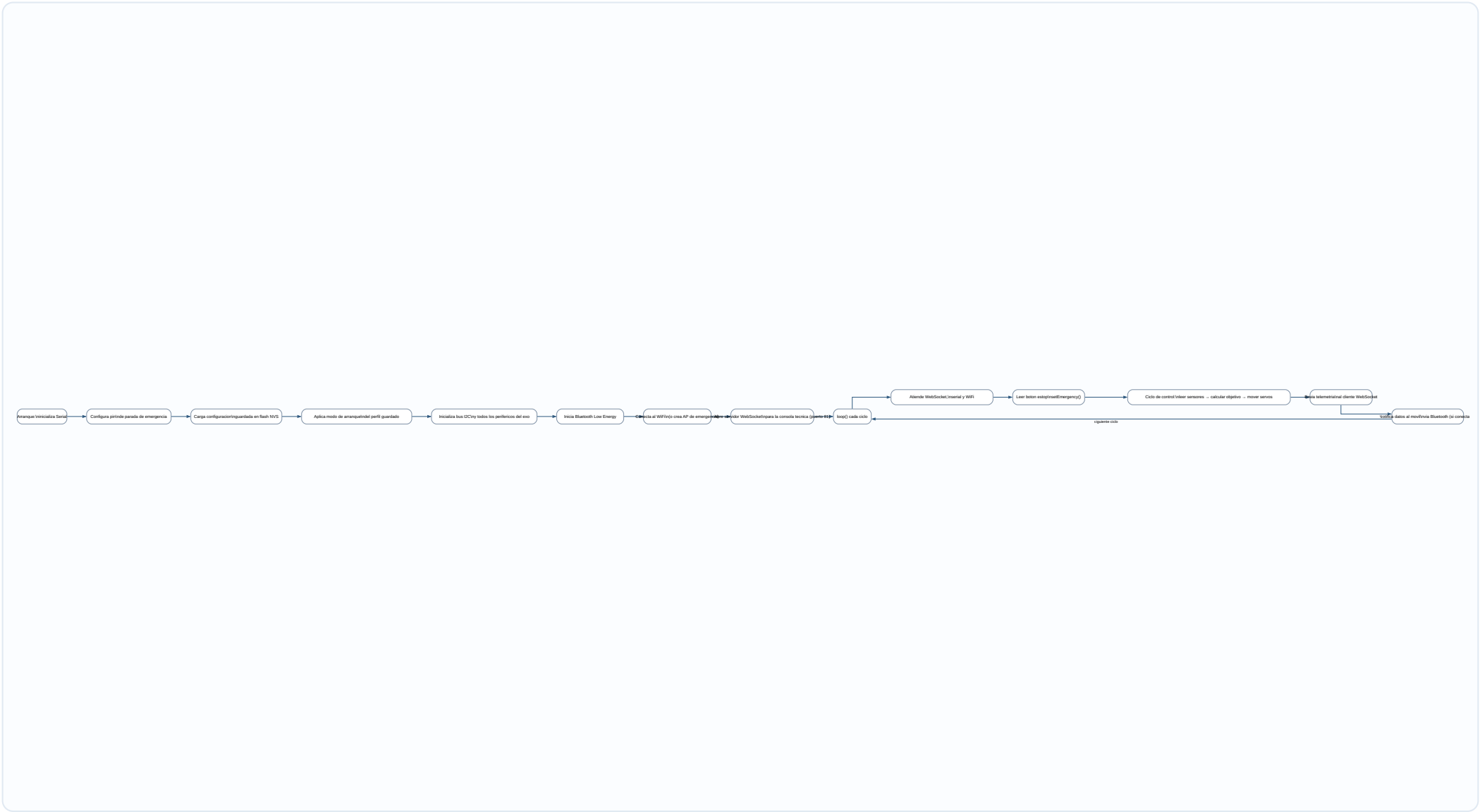
8. Puente Serial Y Eventos SSE

El puente serial permite al navegador comunicarse con un puerto COM físico sin instalar drivers adicionales. Al llamar `POST /api/serial/connect`, `handleSerialConnect()` valida que el puerto exista y lanza `startSerialBridge()`, que abre `arduino-cli monitor` como proceso hijo. Los datos que llegan por `stdout/stderr` se normalizan línea a línea y se retransmiten a todos los navegadores suscritos vía `broadcastSerial()/sendSse()`. Para enviar texto al dispositivo, `POST /api/serial/send` escribe al `stdin` del proceso. Un segundo bucle independiente, `startPortWatcher()/runPortWatcherTick()`, escanea los puertos COM disponibles en segundo plano: si aparece uno nuevo emite el evento `port-added` a todos los clientes SSE, lo que hace que la UI muestre una alerta de "nuevo dispositivo detectado". `stopSerialBridge()` limpia el proceso hijo cuando se desconecta. `GET /api/serial/events` abre el stream SSE al que el navegador se suscribe con `connectSerialEvents()`.



9. ESP32-S3: Setup Y Loop Principal

El firmware del ESP32-S3 arranca en `setup()`: inicializa Serial 115200, configura el pin de parada de emergencia (PIN_ESTOP) como INPUT_PULLUP, carga todos los parametros guardados en flash NVS con `loadPrefs()/setDefaults()`, aplica el modo de arranque (`assisted/manual/automatic`) con `applyBootBehavior()`, inicializa el bus I2C y los perifericos con `setupI2CAndHardware()` — multiplexor TCA9548A, driver PCA9685, 4 IMUs MPU6050 y 2 encoders AS5600 —, arranca el stack Bluetooth Low Energy con `setupBle()` (solo si BLE_ENABLED=1), conecta al WiFi en modo STA o crea un punto de acceso de emergencia con `connectWifi()`, y abre el servidor WebSocket en el puerto 81 para la consola tecnica con `startWebSocketServer()`. El `loop()` corre indefinidamente: `ws.loop()/readSerialCommands()/wifiKeepalive()` atienden comunicaciones, lee el boton de emergencia cada ciclo, y cada CTRL_MS ejecuta el ciclo de control completo (sensores → objetivos → servos) y cada SEND_MS envia telemetria por WebSocket y BLE con `sendData()/sendBleTelemetry()`.



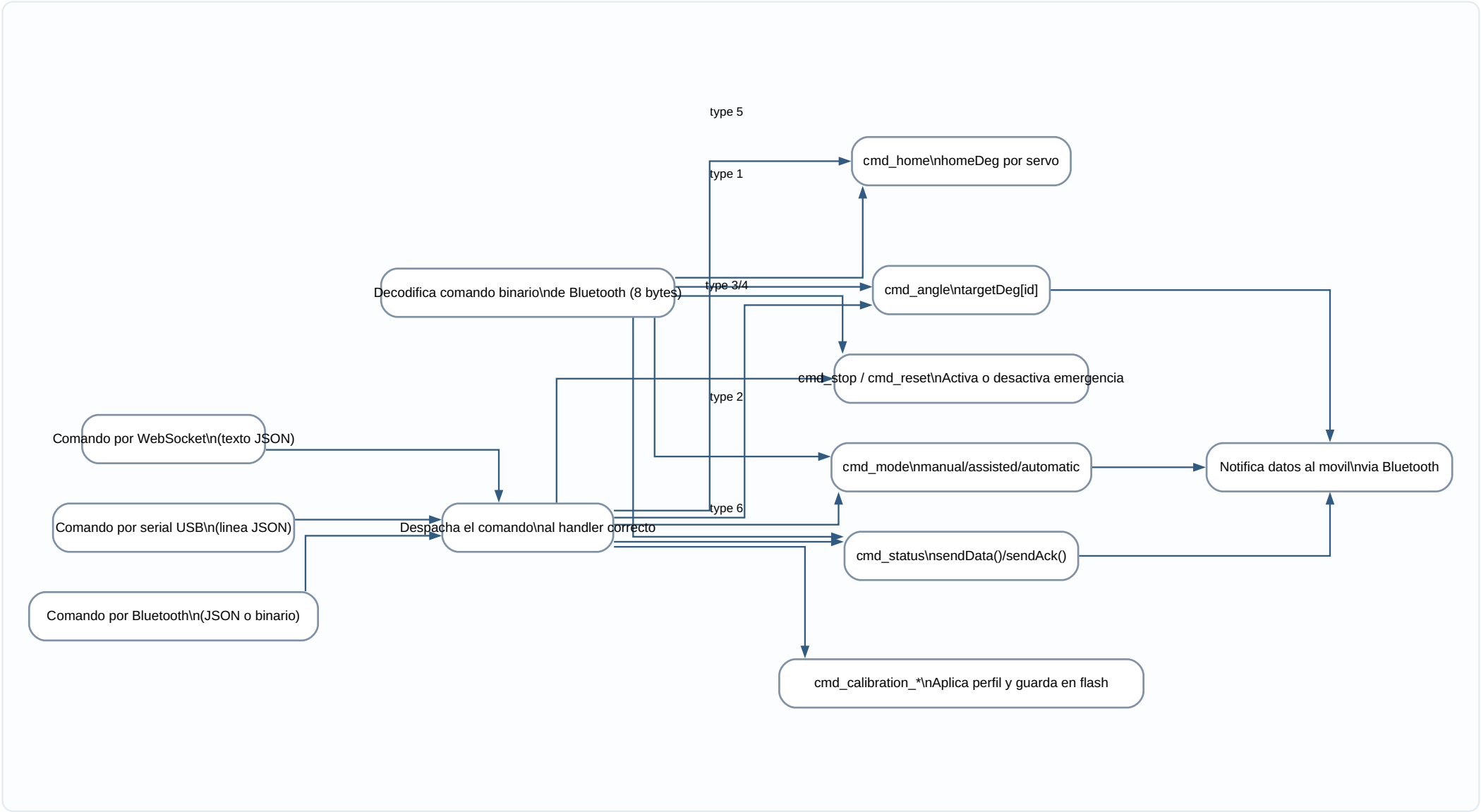
10. ESP32-S3: Sensores, Servos Y Seguridad

El ciclo de control empieza con `readAllSensors()`: la tabla `SENSOR_LINKS` mapea cada servo a su sensor — `readImuDeg()` para los 4 servos de hombro (usa `tcaSel()` para seleccionar el canal del multiplexor TCA9548A y `getMotion6()` para leer el MPU6050, aplicando un filtro complementario que combina acelerometro y giroscopio), y `readAs5600Deg()` para los 2 servos de codo (lee el encoder magnetico AS5600 en canales TCA 5 y 6, aplica filtro EMA y deadband para suavizar). `sensorOnline[]` registra que sensores tienen lectura valida. `updateTargetsFromSensors()` calcula el angulo objetivo de cada servo segun el modo activo: en asistido sigue los movimientos del usuario; en automatico ejecuta secuencias programadas; en manual acepta angulos de la consola. `clampServoDeg()` limita el angulo a los rangos mecanicos seguros configurados en el perfil. `moveServoTowardTarget()` avanza el servo a `maxSpeedDegSec` grados/segundo para movimiento suave. `angleToPwm()/servoPcaChannel()/pca.setPWM()` convierten grados a microsegundos de pulso PWM y lo envian al PCA9685. Si `setEmergency(true)` esta activo, bloquea toda salida PWM inmediatamente.



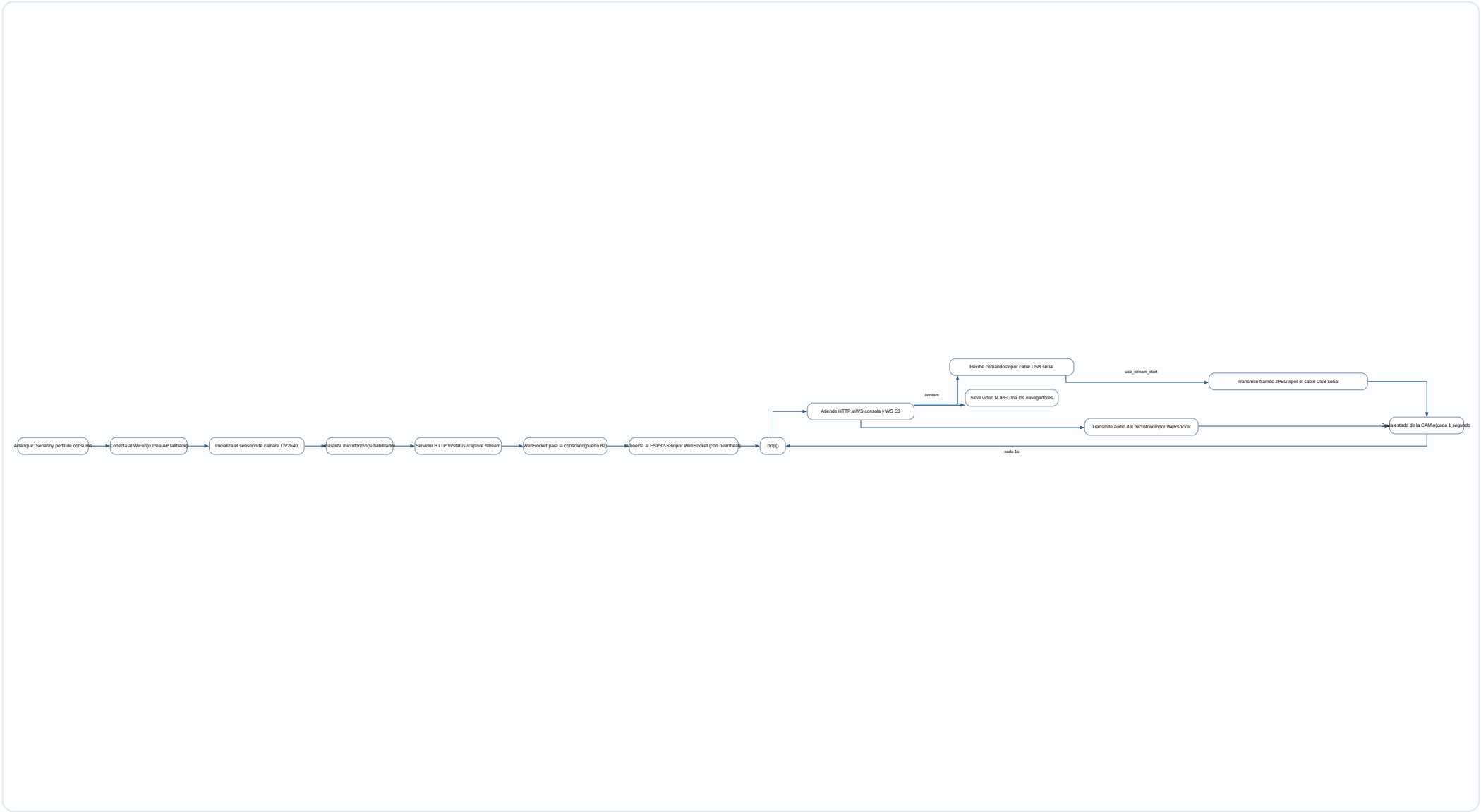
11. ESP32-S3: Comandos JSON Y BLE

El ESP32-S3 acepta comandos desde tres fuentes simultaneas: wsEvent() por WebSocket (texto JSON desde la consola web), readSerialCommands() por USB serial (linea JSON desde terminal), y processBleCommand() por Bluetooth LE (JSON si empieza con '{', o binario de 8 bytes del protocolo VC v1). Todos los comandos JSON convergen en processCmd(json, client) que los despacha por tipo: cmd_status solicita telemetria inmediata, cmd_mode cambia el modo de operacion (manual/asistido/automatico), cmd_angle fija el angulo objetivo de un servo, cmd_stop/cmd_reset activa o desactiva la parada de emergencia via setEmergency(), cmd_home lleva todos los servos a su posicion de reposo, y cmd_calibration_* aplica y guarda el perfil en NVS con applyCalibrationProfile()/savePrefs(). Los comandos binarios BLE (processBleBinaryCommand(), 8 bytes, protocolo VC v1) se decodifican por campo tipo: 1=angulo directo, 2=modo, 3/4=seguridad, 5=home, 6=status. Las respuestas se envian de vuelta por WebSocket y adicionalmente por sendBleTelemetry() como notificacion compacta al movil BLE conectado.



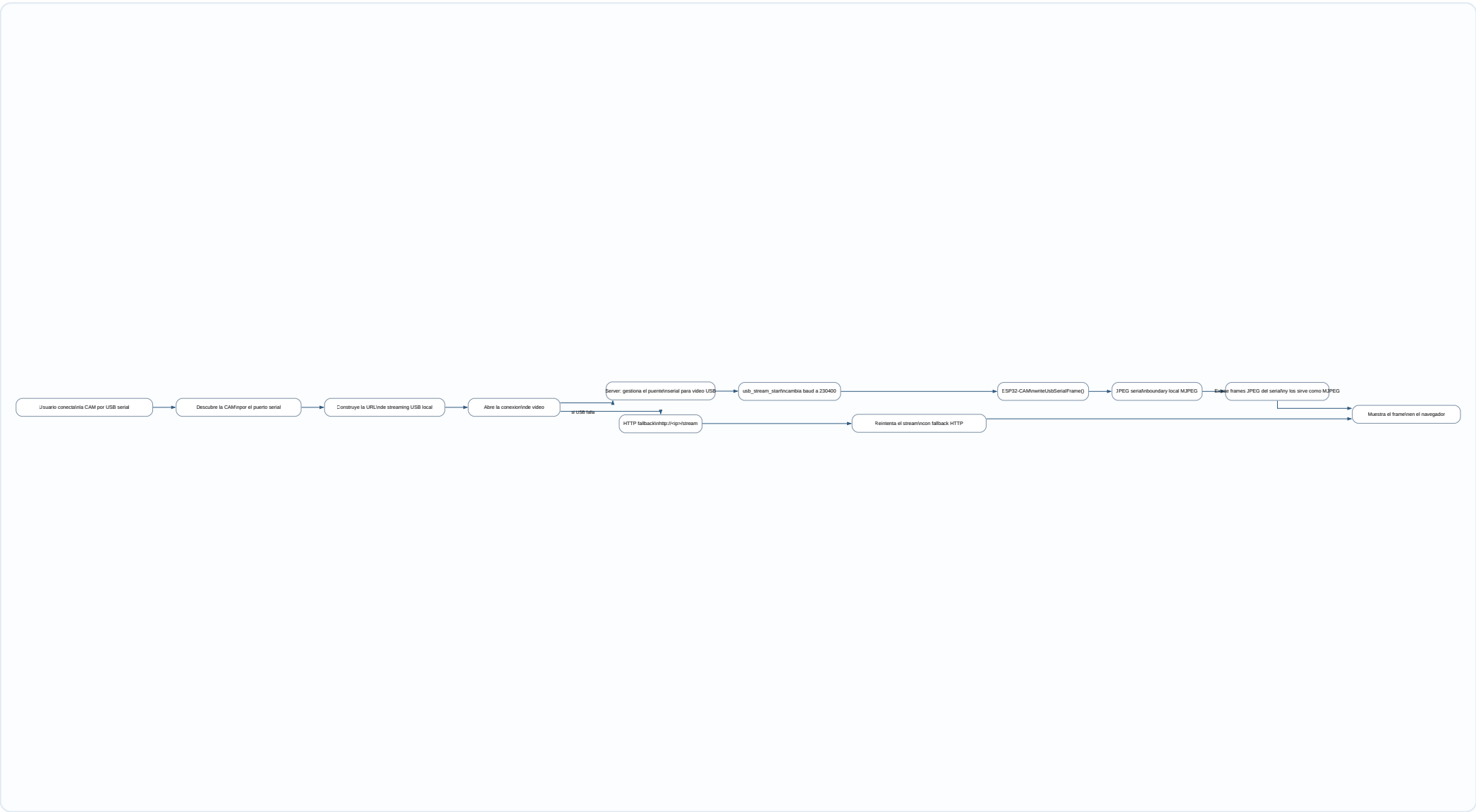
12. ESP32-CAM: Setup, Video, Audio Y Puente Al S3

La ESP32-CAM corre un firmware independiente del S3. En `setup()`: inicializa Serial y configura el perfil de consumo, conecta al WiFi STA o crea AP de emergencia con `connectWifi()`, inicializa el sensor de camara OV2640 con `setupCamera()`, el microfono I2S con `setupMic()` (si habilitado en config), arranca el servidor HTTP en puerto 80 con `setupHttp()` (rutas: `/`=pagina de status, `/status=JSON`, `/capture=foto JPEG unica`, `/stream=video MJPEG`), abre un WebSocket de "app" en puerto 82 con `appWs.begin()` para recibir comandos de la consola, y se conecta como cliente WebSocket al ESP32-S3 en puerto 81 con `s3Ws.begin(S3_HOST, 81)` para reenviarle el estado de la camara. En `loop()`: `http.handleClient()/appWs.loop()/s3Ws.loop()` atienden peticiones; `handleStream()` sirve video MJPEG multipart por HTTP; `readAndStreamAudio()` transmite audio PCM16 mono por WebSocket; `readSerialCommands()` acepta los comandos `cmd_status` y `usb_stream_start` desde la PC; `writeUsbSerialFrame()` transmite frames JPEG por el puerto serial USB usando el marcador binario `VESTA_USB_MJPEG_BEGIN` para que `server.js` los reconstruya. Cada 1 segundo `sendStatus()` notifica el estado de la CAM a la app y al S3.



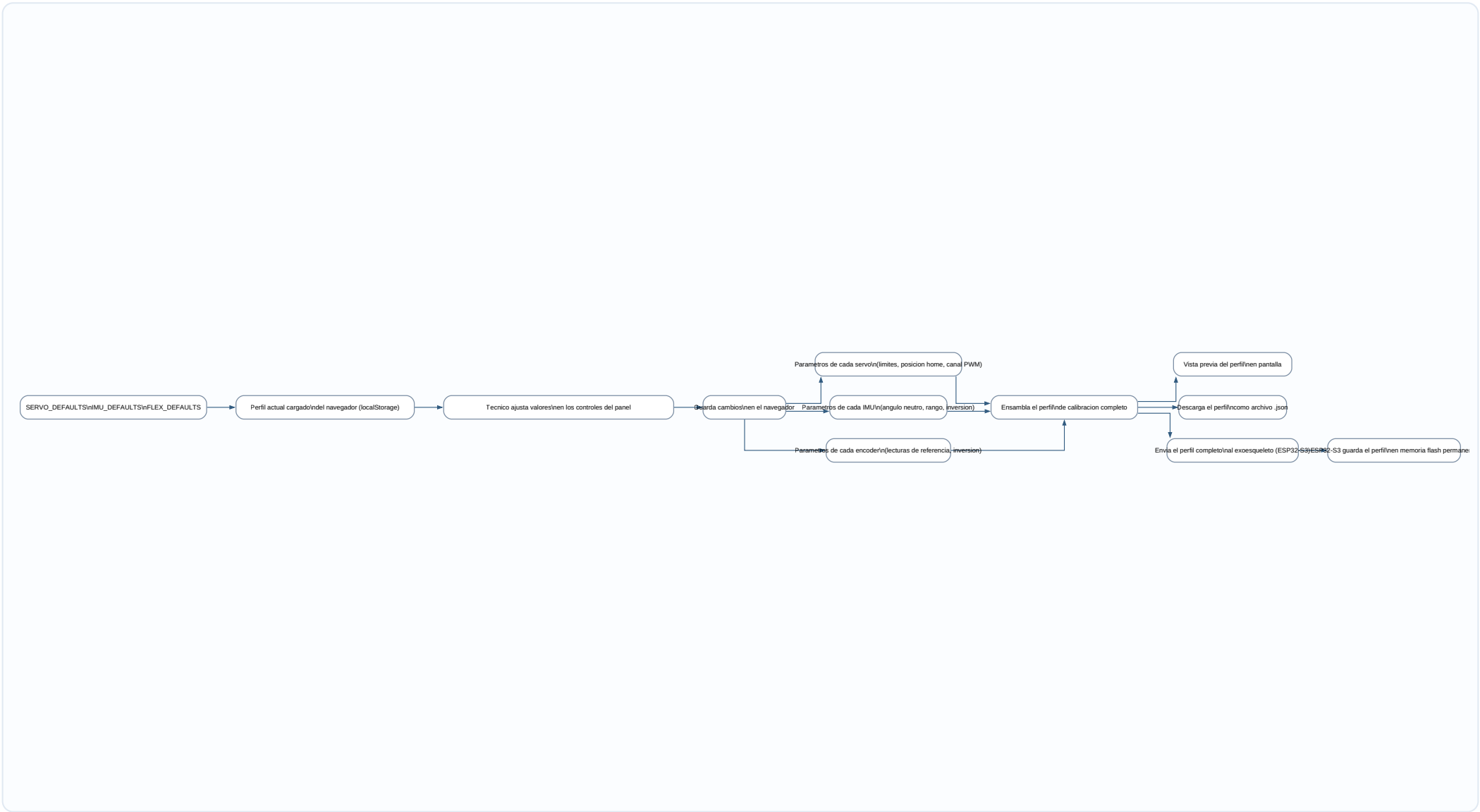
13. Flujo De Video En Banco: USB MJPEG Y Fallback HTTP

Cuando el tecnico esta en una red diferente a la del exoesqueleto (caso tipico en banco de trabajo), la consola recibe video de la ESP32-CAM por el cable USB serial sin necesidad de conexion WiFi directa. Flujo completo: el usuario pulsa "Conectar CAM USB" que llama connectCamManualCom(); la app hace POST /api/cam/discover que ejecuta readCamDiscoverySerial() — abre el COM de la CAM, envia cmd_status y lee la IP/estado de vuelta —; localCamUsbStreamUrl() construye la URL del endpoint de streaming USB local; startCamera()/openCameraStream() inician el stream. handleCamUsbStream() en server.js lanza un proceso PowerShell hijo que maneja el puerto serial, envia el comando usb_stream_start haciendo que la CAM suba a 230400 baud, y la ESP32-CAM empieza a serializar frames JPEG con writeUsbSerialFrame() delimitados por VESTA_USB_MJPEG_BEGIN. Del lado del servidor, pumpCameraMjpeg()/extractJpegFrame() leen el stream serial y construyen un MJPEG HTTP local que el navegador consume como si fuera una camara IP normal. renderCameraFrame() crea un objectURL para cada frame y lo muestra en el . Si el USB falla, hay fallback automatico a HTTP directo (http://IP/stream), con reintentos y proxy gestionados por recoverCamStreamAfterImageError().



14. Perfil De Calibracion Y Persistencia

El perfil de calibracion contiene todos los parametros mecanicos y de sensor necesarios para adaptar el exoesqueleto a un paciente especifico. Flujo: los valores por defecto parten de SERVO_DEFAULTS/IMU_DEFAULTS/FLEX_DEFAULTS definidos en el firmware; el estado actual se carga de localStorage del navegador (clave STORAGE_KEY) para persistir entre sesiones. El tecnico ajusta valores en los inputs del panel — onServoInput() para limites y offset de cada servo, onImuInput() para la calibracion del IMU, onFlexInput()/updateTuningFromControls() para los parametros de asistencia —; cada cambio llama markDirty()/saveLocal() para no perder el trabajo. buildProfile() ensambla el objeto completo con tres subobjetos: profileServos() (min/max/home/direccion/canal PWM/offset), profileImus() (angulo neutro/min/max/inversion de eje) y profileFlex() (lectura adc0/adc90/neutral/inversion). Desde buildProfile() hay tres salidas posibles: updateProfilePreview() muestra el JSON en pantalla para inspeccion, downloadProfile() exporta el perfil como archivo .json, y sendFullProfile() envia el comando cmd_calibration_profile al ESP32-S3 por WebSocket, donde applyCalibrationProfile() lo aplica en caliente y savePrefs() lo guarda en la memoria flash no volatil NVS para que persista tras reinicios.



15. Readiness, Pruebas Y Seguridad Operativa

Antes de entregar el exoesqueleto al paciente, la consola debe pasar una batería de pruebas automaticas definidas en TEST_DEFS. runAllTests() las ejecuta en secuencia: cada prueba usa awaitPacket() para esperar un paquete de telemetria reciente del S3 y verifica: (1) Prueba de enlace — envia cmd_status y confirma respuesta del S3; (2) Prueba de modos — cambia entre manual/asistido/automatico y verifica que el S3 confirme cada cambio; (3) Prueba de servos — envia cmd_angle a cada servo y comprueba que el angulo reportado coincida; (4) Prueba de sensores — verifica que todos los IMU MPU6050 (hombros) y AS5600 (codos) esten marcados como online en sensorOnline[]; (5) Prueba de emergencia — envia cmd_stop y cmd_reset verificando que el S3 active y desactive setEmergency() correctamente. Los resultados se acumulan en updateTestSummary()/setTestState() y se visualizan en renderReadiness() como checks verdes o rojos por fase. La compuerta "Enviar perfil habilitado?" solo se abre cuando todos los checks estan en verde: entonces sendFullProfile() envia el perfil de calibracion final al S3 y se habilita el modo de entrega. Si hay pruebas pendientes o fallidas, el sistema queda bloqueado visualmente hasta que se corrijan.

